Mechanical Design and Manufacturing

Lightweight construction, articulated mobility, and reliable rescue-kit deployment

3.a Mechanical Design and Manufacturing

The robot was designed to move quickly and precisely through a maze while remaining capable of crossing ramps, steps, and uneven terrain. The mechanical structure combines lightweight LEGO components with purpose-built 3D-printed modules. The resulting design reduces unnecessary mass, maintains wheel contact on difficult surfaces, protects sensitive electronics, and provides controlled bidirectional rescue-kit deployment.

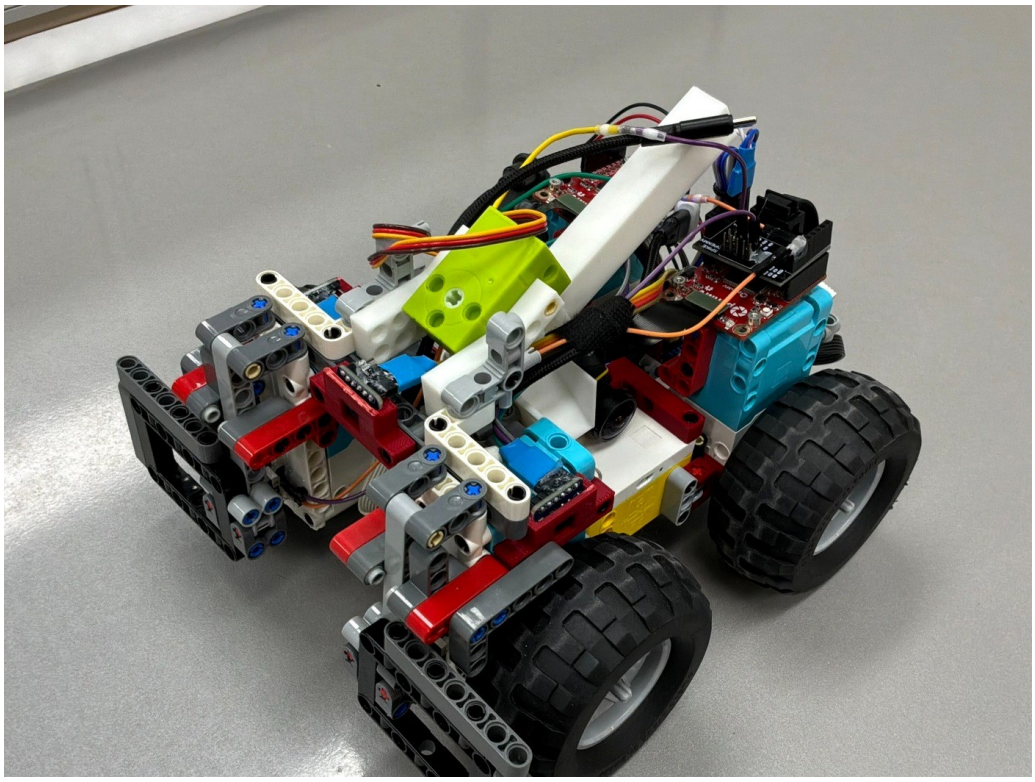

Figure M1. Overall mechanical configuration of the rescue-maze robot.

1. Main Structure

The core frame is constructed primarily from LEGO components. Compared with heavy acrylic or metal plates, the lightweight frame reduces the load on the drive motors and limits forward momentum during braking. This helps the robot stop more accurately at tile boundaries and reduces skidding during direction changes. Custom 3D-printed supports are used only where standard LEGO parts cannot provide the required geometry or reinforcement.

2. Actuators and Power Train

The drive system uses four large LEGO motors and large rubber wheels. The front and rear chassis sections are connected through a central articulated joint so that they can rotate relative to each other. When the robot climbs a ramp or crosses a step, this joint allows the chassis to conform to the terrain instead of forcing the wheels to remain on one rigid plane. This keeps all four wheels in contact with the surface, improves traction, and reduces wheel spin. The large wheel diameter also lowers the risk of the underside becoming trapped on an obstacle edge.

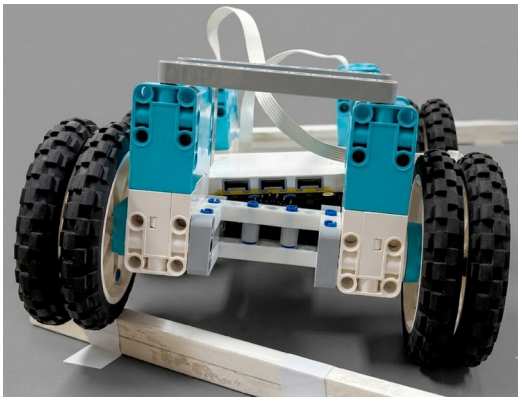

Figure M2. Articulated chassis structure and central pivot.

3. Subassemblies and Modules

Camera Mount Module

The camera mount holds the OpenMV camera securely while preserving a wide viewing angle. It is positioned as far inward as possible to protect the lens and connector from contact with maze walls. The mount is symmetrical to improve alignment and includes cable support to prevent the connector from loosening during movement. Because the wall height is 15 cm, the camera is placed near the mid-height of the wall, approximately 7.5 cm above the floor.

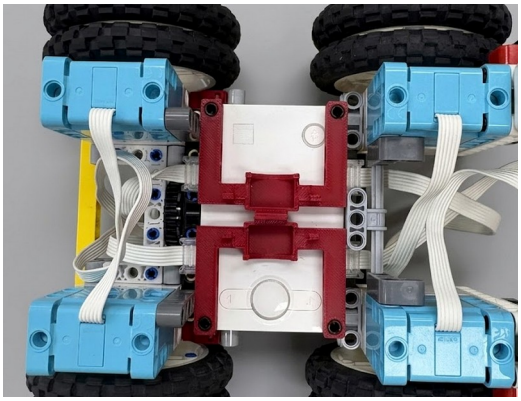

Figure M3. Camera mount geometry and protected lens position.

Rescue-Kit Storage and Deployment Module

The rescue-kit module combines a storage container, left and right deployment ramps, and a release mechanism. The container can hold up to eight rescue kits. The ramps guide each kit toward the selected side of the robot, and the release mechanism dispenses kits individually. This configuration enables accurate placement without rotating the entire robot toward the victim.

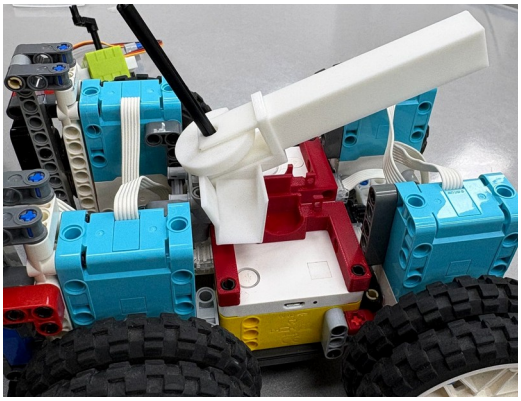

Figure M4. 3D-printed bidirectional rescue-kit ramp.

Distance-Sensor Mount Module

Three distance sensors are mounted at the front and sides of the robot. Their positions provide clear forward, left, and right measurement directions while keeping the sensors mechanically protected. Placing the sensors near the front of the robot increases the available reaction distance before the chassis reaches a wall or obstacle.

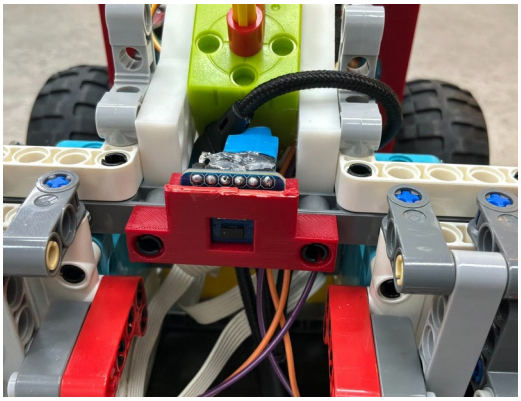

Figure M5. Front and side distance-sensor mounts.

Handle and Structural Reinforcement Module

A dedicated handle allows the robot to be lifted and transported without pulling on cables or sensitive components. The surrounding reinforcement links the handle to the main chassis and helps prevent separation between the chassis sections and the central turntable during repeated obstacle-crossing runs.

4. Rescue-Kit Deployment Mechanism

The bidirectional deployment mechanism was custom-built with 3D printing. Its slide-shaped ramps allow each rescue kit to move downward under its own weight, reducing the need for complex mechanical parts. A small control motor operates the upper release mechanism and selects the left or right deployment direction. Because the robot does not need to rotate its entire body before delivering a kit, the design reduces mission time and simplifies victim servicing in narrow passages.

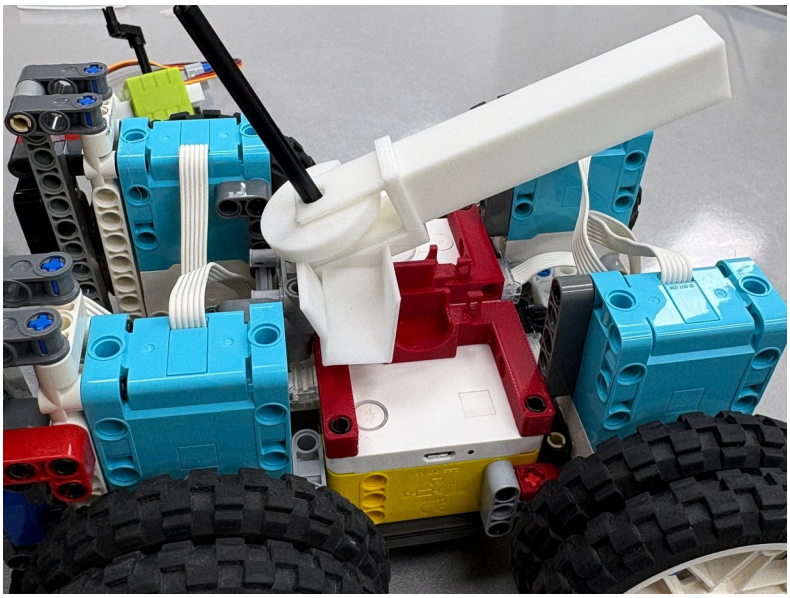

Figure M6. Controlled left-right rescue-kit release structure.

During iterative assembly, the ramp angle, opening size, and release speed were adjusted to prevent a kit from catching on the printed structure. The final design emphasizes gravity-assisted motion, a short release path, and a mechanically simple gate. These choices improve repeatability while keeping the mechanism compact.

5. Mechanical Validation and Iteration

Validation Item	Purpose	Design Response
Braking and turning	Check whether the robot stops accurately at tile boundaries and avoids excessive skidding.	Reduced mass and large rubber wheels improve stopping control and grip.
Ramp and step crossing	Confirm that the wheels maintain contact with uneven surfaces.	The articulated chassis allows front and rear sections to adapt independently.
Camera stability	Check lens alignment and cable security during repeated movement.	The inward-mounted, reinforced camera holder protects the lens and connector.
Kit deployment	Verify one-by-one release to either side without rotating the chassis.	The gravity-assisted ramp and controlled gate reduce jamming risk.

Electronic Design and Manufacturing

Controller, sensor network, power distribution, and actuator interfaces

3.b Electronic Design and Manufacturing

The electronic system is divided between the LEGO SPIKE Prime Hub and the OpenMV H7 vision-and-sensor subsystem. The SPIKE Prime Hub is responsible for drive control, IMU-based motion correction, and high-level navigation. The OpenMV H7 processes visual targets, reads three VL53L4CD time-of-flight distance sensors and two touch sensors, and receives commands for the rescue-kit release motor. A PUPRemote interface links the two devices so that sensor data and deployment commands can be exchanged during autonomous operation.

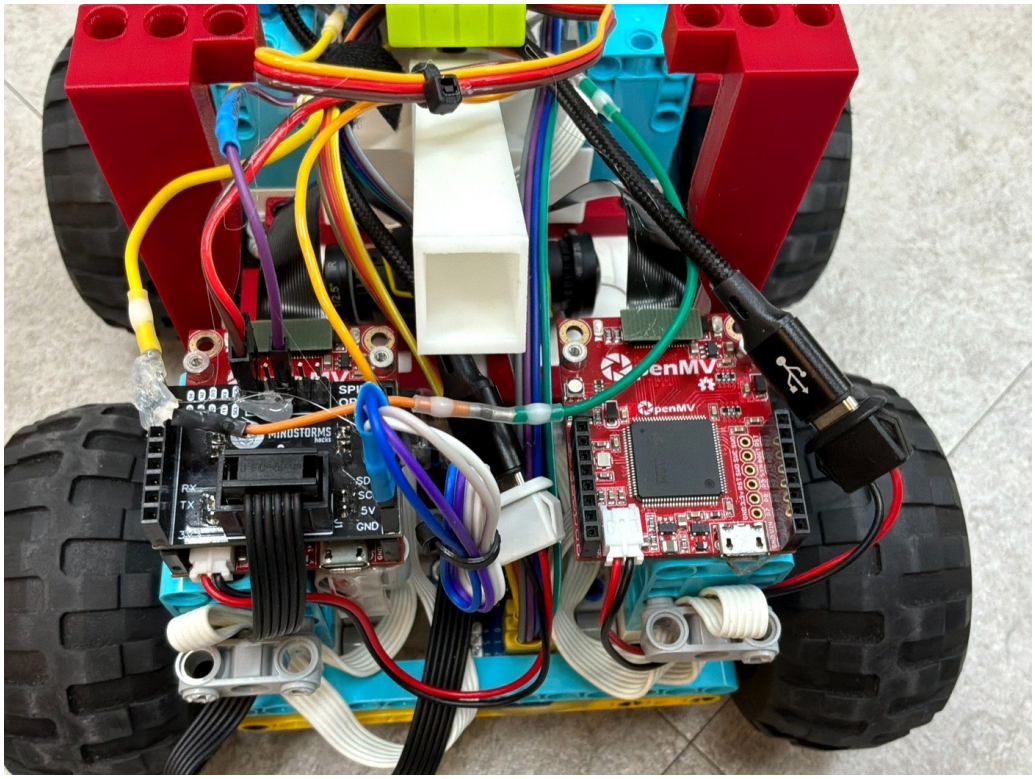

Figure E1. Top view of the controller, OpenMV H7 board, wiring, and interface modules.

1. Main Controller and Actuators

The LEGO SPIKE Prime Hub serves as the main controller. Four LEGO drive motors provide the traction required to move the large off-road wheels and cross ramps or steps. The hub also uses its built-in IMU to maintain the desired heading and reduce drift during straight driving and turning. A separate small motor controls the rescue-kit release mechanism so that kits can be dispensed accurately in the required direction.

Electronic Component	Primary Role
LEGO SPIKE Prime Hub	Main navigation controller, motor control, coordinate mapping, and built-in IMU processing.
Four LEGO drive motors	Independent traction for the four large wheels and stable obstacle crossing.
Small deployment motor	Controls the rescue-kit release gate and the selected deployment direction.
OpenMV H7	Vision processing, distance-sensor acquisition, touch-sensor input, and communication with the hub.

2. Sensors Used - Vision, IMU, Distance, and Touch

Vision Sensor: One OpenMV H7 camera is mounted inside the chassis with its lens directed toward the wall area. The camera identifies concentric cognitive targets and the Greek-letter victims Φ, Ψ, and Ω. The protected inward position limits mechanical interference while maintaining a useful field of view.

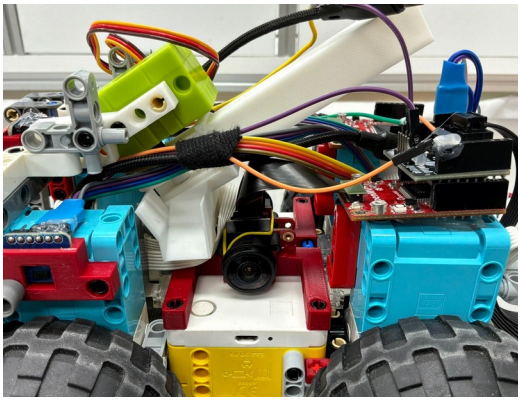

Figure E2. OpenMV H7 board and protected camera position.

Distance Sensors: Three VL53L4CD time-of-flight sensors measure the distance to the left, front, and right sides of the robot. These readings support wall detection, wall-following correction, and collision avoidance. The sensor mounts are aligned to preserve clear measurement directions near the front of the chassis.

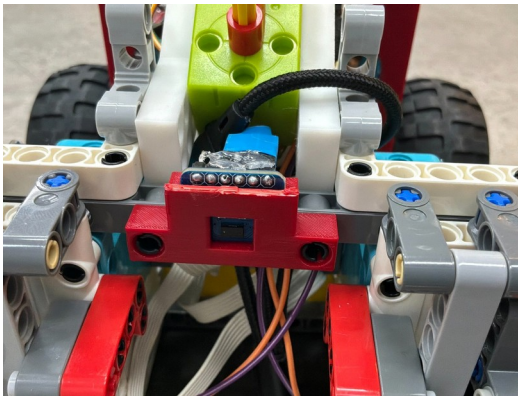

Figure E3. Forward and side distance-sensor arrangement.

IMU and Touch Sensors: The built-in SPIKE Prime IMU is used for heading correction and turn control. Two touch sensors provide a mechanical backup for close-range contact and wall alignment. If a distance reading is insufficient during a close approach, the touch sensors allow the robot to stop and correct its position.

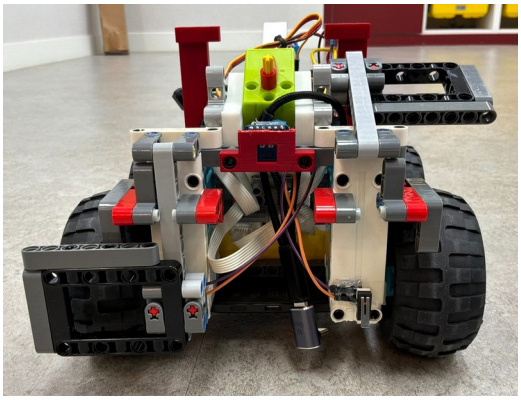

Figure E4. Front structure containing the protected contact-sensing area.

3. Power Subsystem

The robot uses the SPIKE Prime Hub battery together with an auxiliary external battery pack for the additional electronics. Separating the main controller supply from the external vision-and-sensor load helps maintain stable operation during longer missions and during high-load actions such as ramp climbing, obstacle crossing, and rescue-kit deployment. The cable routing is fixed to the chassis to reduce connector strain during repeated movement.

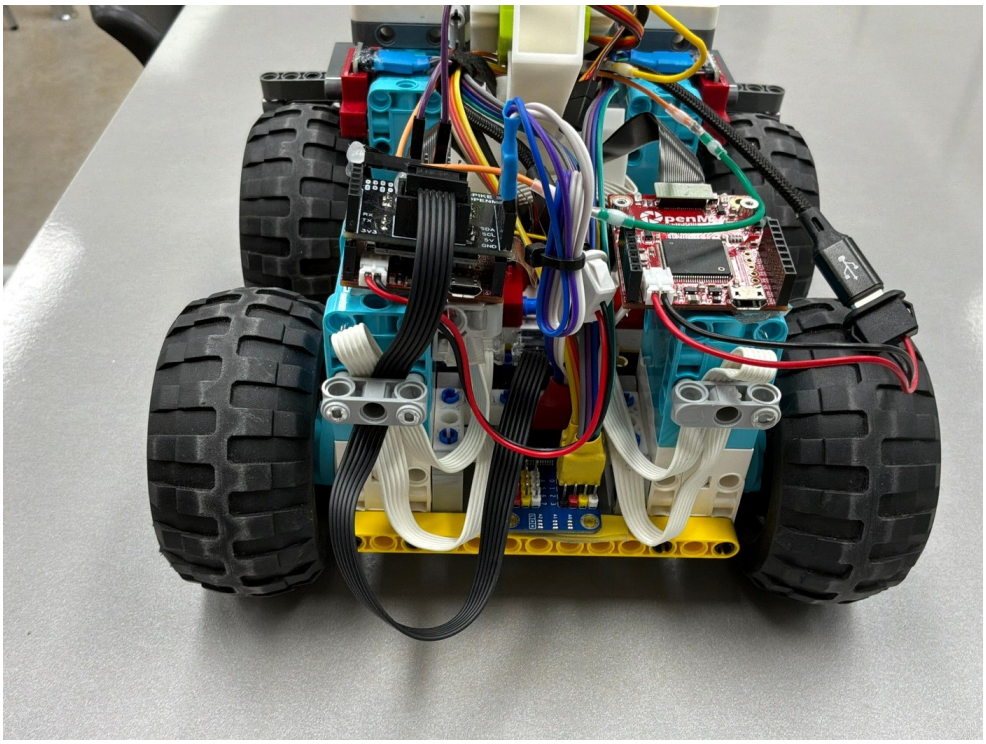

Figure E5. Rear view of the electronic subsystem, interface board, and cable routing.

4. Communication and Integration

PUPRemote is used to connect the OpenMV-based subsystem to the SPIKE Prime Hub. OpenMV operates as an external Powered Up sensor. It sends the three distance readings, touch-sensor states, and victim-detection results to the hub. After the hub determines the required rescue-kit quantity and direction, it returns a deployment command to OpenMV. This bidirectional structure keeps navigation decisions in the main controller while allowing the vision subsystem to continue camera processing and sensor acquisition.

Data Flow	Transferred Information
OpenMV H7 -> SPIKE Prime Hub	Left/front/right ToF distances, two touch-sensor states, cognitive-target score, and letter-victim result.
SPIKE Prime Hub -> OpenMV H7	Rescue-kit quantity and deployment-direction command.
OpenMV H7 -> Deployment Motor	Queued motor operation for controlled kit release while sensing continues.

5. Electronic Validation and Iteration

Validation Item	Verification Focus	Design Response
Distance sensing	Confirm stable left/front/right wall measurements during movement.	Rigid front-and-side mounting maintains consistent sensor alignment.
Vision processing	Check victim recognition while the robot is not perfectly centered.	The camera is protected and positioned to preserve a wide usable view.
Contact backup	Verify safe response during close wall approaches.	Two touch sensors provide a mechanical fallback for stopping and correction.
Power stability	Check operation during driving, obstacle crossing, and kit release.	The auxiliary battery pack supports external electronics and reduces supply fluctuation.
Communication	Confirm that sensing continues while deployment commands are executed.	Bidirectional PUPRemote communication separates data acquisition from actuation commands.

ROBOCUPJUNIOR RESCUE MAZE 2026

TEAM DESCRIPTION PAPER - SOFTWARE

General Software Architecture

Distributed sensing, coordinate-based navigation, and non-blocking rescue-kit control

4.a General Software Architecture

The software is divided between a LEGO SPIKE Prime Hub and an OpenMV H7 camera board. The SPIKE Prime Hub runs **main.py** and controls locomotion, heading correction, wall-map construction, route selection, victim-position storage, and return-to-start navigation. OpenMV runs **tof_circle.py** and combines camera analysis, three VL53L4CD ToF sensors, two touch sensors, and the rescue-kit dispenser interface. The two controllers communicate through PUPRemote, allowing OpenMV to behave as an external SPIKE-compatible sensor while still performing time-sensitive vision and ranging tasks independently.

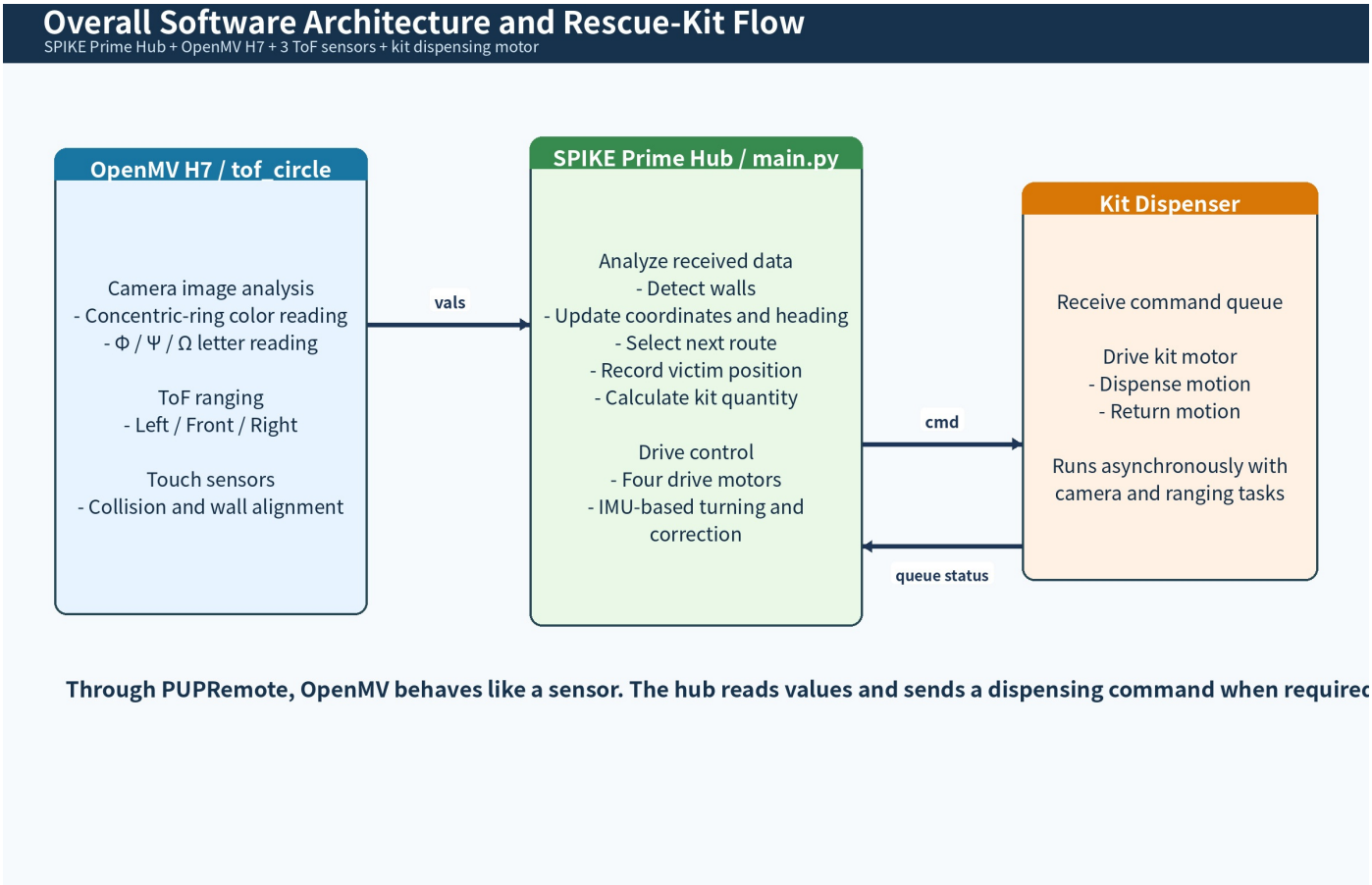

Figure S1. Distributed software architecture and rescue-kit command flow.

Program Responsibilities

Program or library	Role in the integrated system
main.py	Runs on the SPIKE Prime Hub. Controls four drive motors, reads the Hub IMU, updates the coordinate map, records walls and victims, selects routes, calculates kit quantity, and returns to start.
tof_circle.py	Runs on OpenMV H7. Performs camera-based target recognition, polls three ToF sensors, reads two touch sensors, transmits a compact sensor packet, and executes dispenser commands through a queue.
pupremote.py / lpf2.py	Provides the data channel and command channel between SPIKE Prime and OpenMV, allowing a third-party controller to behave as a Powered Up sensor.
vl53l4cd.py / i2c_device.py	Initializes and reads the VL53L4CD distance sensors connected through an I2C multiplexer.

Data Flow and Integration Strategy

OpenMV continuously updates a compact packet containing the cognitive-target flag and score, letter-victim flag and score, left/front/right ToF values, and two touch-sensor states. The Hub reads this packet during driving, turning, and alignment. When a new victim is detected, the Hub stops the robot, records the victim coordinate, converts the classification score into a rescue-kit quantity, and sends one or more commands back to OpenMV. OpenMV places each command into a queue so that dispenser motion does not block camera analysis or distance sensing.

Integration Problems and Software Responses

Integration problem	Software response	Result
Limited Hub ports and heterogeneous sensors	OpenMV aggregates camera, three ToF sensors, and two touch sensors, then exposes the combined values through PUPRemote.	A single Hub-side connection carries the external sensor data and command channel.
Vision, ranging, and dispenser movement can compete for processing time	OpenMV separates vision, ToF polling, sensor updates, and drop motion with uasyncio and a bounded command queue.	Kit deployment proceeds without unnecessarily delaying recognition and ranging.
Simple wall following can repeatedly revisit dead ends	Each tile stores walls, visit count, exit-use count, dead-end state, and victim information. Exhausted branches are propagated as dead ends.	The robot reduces unnecessary revisits while preserving an intuitive exploration priority.
A final return route is not guaranteed by local decisions	The Hub runs Breadth-First Search over known open passages and converts the first absolute direction into a relative movement command.	The robot can return to the start tile (0, 0) after exploration.
Victim detections can occur while crossing between tiles	Wheel rotation is compared with half of the tile-travel distance to assign the detection to the current tile or the next tile.	Victim coordinates remain consistent with the map.

Coordinate-Based Exploration and Return to Start

The maze is represented as a graph of (x, y) coordinates rather than a simple movement history. Heading is stored as an absolute direction: North 0, East 1, South 2, and West 3. After every valid tile movement, the program updates coordinates and links the open passage in both directions. During exploration, it preserves a right-forward-left preference but prioritizes exits and neighboring tiles used less often. After the exploration limit, BFS identifies the first direction toward the start tile.

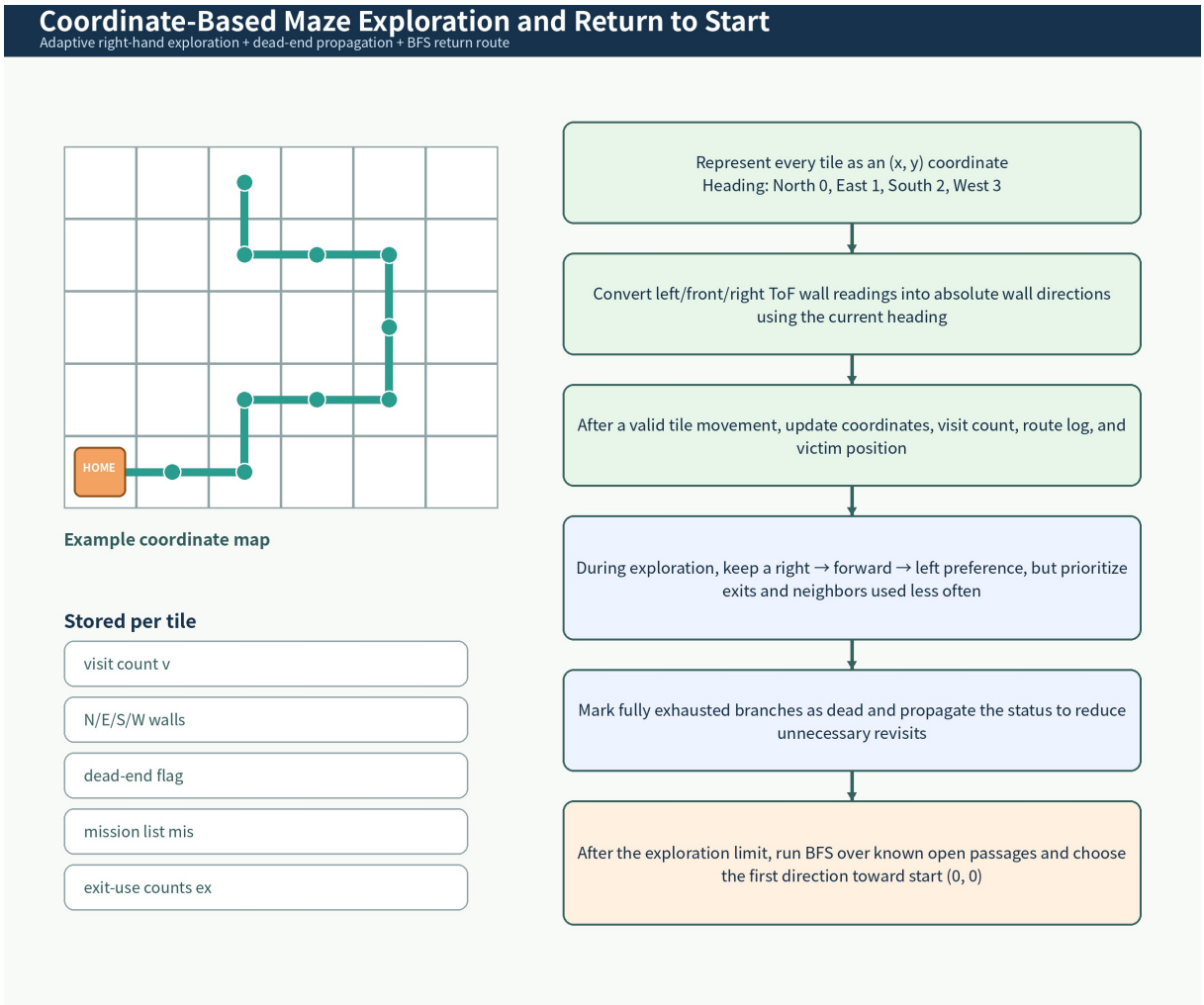

Figure S2. Coordinate-map generation, adaptive exploration, dead-end propagation, and BFS return.

Tools and Algorithms Used

Tool or algorithm	Purpose	Why it was selected
Pybricks on SPIKE Prime	Drive-motor control, IMU-based correction, heading management, and route logic.	Direct hardware access and deterministic control loops.
OpenMV MicroPython	Image capture, lens correction, LAB-threshold classification, binarization, and blob analysis.	Runs directly on the camera board with low overhead.
PUPRemote and LPF2	Bidirectional Hub-OpenMV communication.	Makes OpenMV appear as an external SPIKE sensor while preserving a return command path.
I2C multiplexer + VL53L4CD driver	Sequential reading of left, front, and right distance sensors.	Allows three same-address ToF sensors to share one bus.
Coordinate graph + dead-end propagation	Stores walls, visits, route use, and victim positions.	Improves exploration efficiency compared with an unmodified right-hand rule.
Breadth-First Search	Finds a return direction over known open passages.	Produces a shortest known route back to start.
AI models	Not used. Recognition is rule-based.	Deterministic image processing was selected for repeatability, explainability, and speed on embedded hardware.

Software-Level Validation Parameters

The following values are currently implemented in the integrated code. They make the software behavior reproducible. Measured full-course success rates can be added after repeated field trials.

Item	Implemented value	Purpose
Hub-OpenMV sensor packet	9 values: bbbbBBBbb	Compact transmission of recognition, ToF, and touch information.
ToF polling interval	Adaptive range: 10-80 ms	Maintains ranging updates while recovering from temporary sensor delays.
Dispenser queue	Maximum 4 pending commands	Prevents repeated kit commands from blocking vision and ranging tasks.
Return navigation	BFS over known open passages	Calculates the first movement toward start tile (0, 0).
Victim duplicate handling	One handled record per coordinate	Avoids repeated kit deployment at the same victim location.

Runtime Control Sequence

Step	Hub and OpenMV behavior
1. Read values	The Hub reads the latest recognition, ToF, and touch values through the PUPRemote values channel during driving, turning, and wall alignment.
2. Update map	The Hub converts left/front/right sensor readings into absolute wall directions using the current heading, then updates the current coordinate and neighboring passage links.
3. Select route	The route selector evaluates right, forward, left, and back options using wall states, dead-end flags, exit-use counts, and neighboring-tile visit counts.
4. Handle victim	A newly detected victim is assigned to the current or next tile using wheel rotation, stored once per coordinate, and converted into a kit quantity.
5. Deploy kit	The Hub sends one or more dispenser commands. OpenMV queues the commands and executes the release sequence asynchronously.
6. Return home	After exploration, BFS searches known open passages and returns the first absolute direction toward (0, 0).

Sensor Packet Definition

Field group	Values	Use
Recognition	circle flag, circle score, text flag, text score	Victim classification and kit quantity calculation.
Distance	left, front, right ToF values	Wall detection, wall following, and route decisions.
Contact	left and right touch states	Collision recovery and wall alignment backup.

Document note: This section describes the software currently implemented in main.py and tof_circle.py. No source code is included in the TDP section.

ROBOCUPJUNIOR RESCUE MAZE 2026

TEAM DESCRIPTION PAPER - SOFTWARE

Innovative Solutions

Rule-based embedded vision, asynchronous sensor processing, and adaptive maze exploration

4.b Innovative Solutions

The robot uses several custom software approaches to solve the Rescue Maze challenge on compact embedded hardware. Rather than relying on a general-purpose AI model, the system combines deterministic image processing, asynchronous sensor tasks, a coordinate-based map, and a non-blocking rescue-kit command queue. The algorithms are explainable and tunable while remaining fast enough for autonomous navigation.

1. Five-Zone Cognitive-Target Ring Detection

A cognitive target contains five concentric color zones. Averaging the entire target would destroy the information contained in individual rings, especially when neighboring zones share a color or lighting is uneven. The program estimates the target center and radius, samples five radii independently, and uses majority voting at each radius. Each representative ring color is converted into a numerical value and the five values are summed before transmission to the Hub.

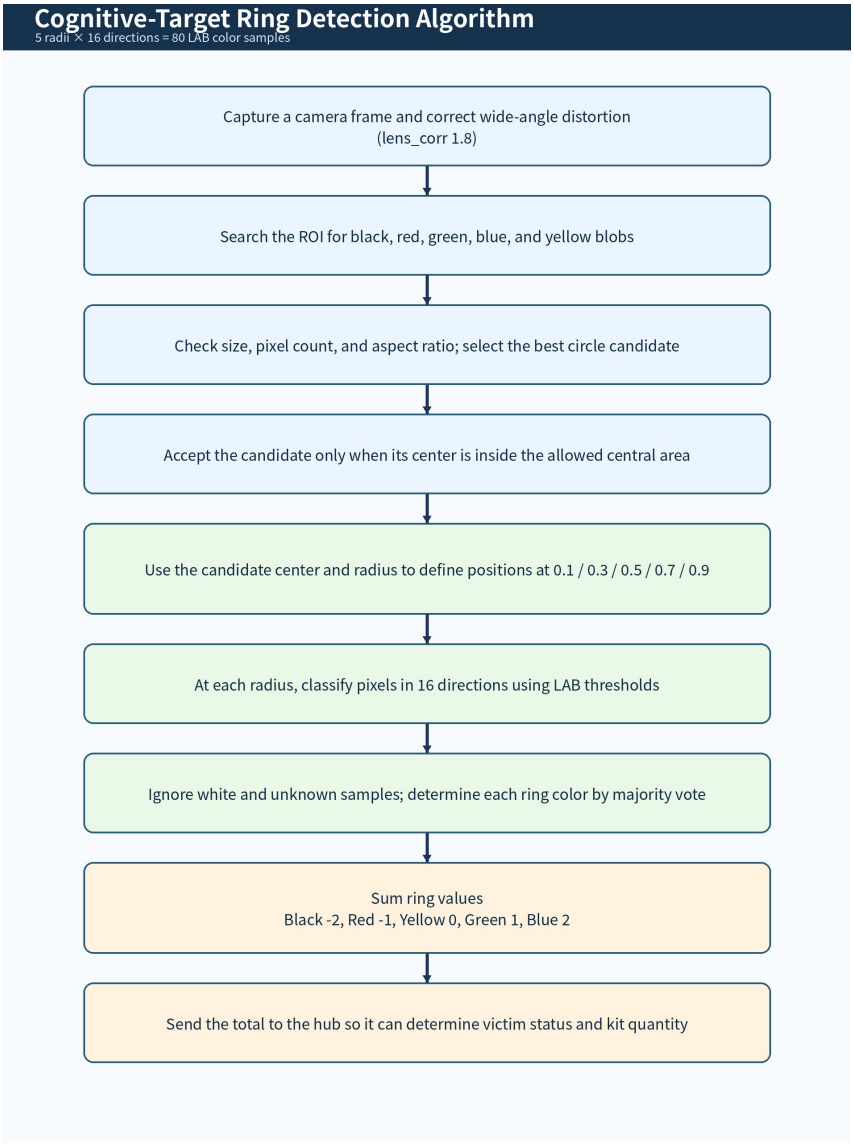

Figure S3. Cognitive-target detection using 5 radii x 16 directions = 80 LAB color samples.

Color	Value	Meaning in ring sum
Black	-2	Negative contribution
Red	-1	Negative contribution
Yellow	0	Neutral
Green	1	Positive contribution
Blue	2	Positive contribution

2. Rule-Based Letter-Victim Recognition

When no circle candidate is present, OpenMV enters the letter-recognition branch. The image is converted to grayscale, smoothed, and binarized. The program selects a centered candidate, estimates the line angle, applies rotation correction, and reduces the internal structure to a 3 x 3 pattern. The classification is stabilized with recent-frame voting, reducing errors caused by camera motion and momentary noise.

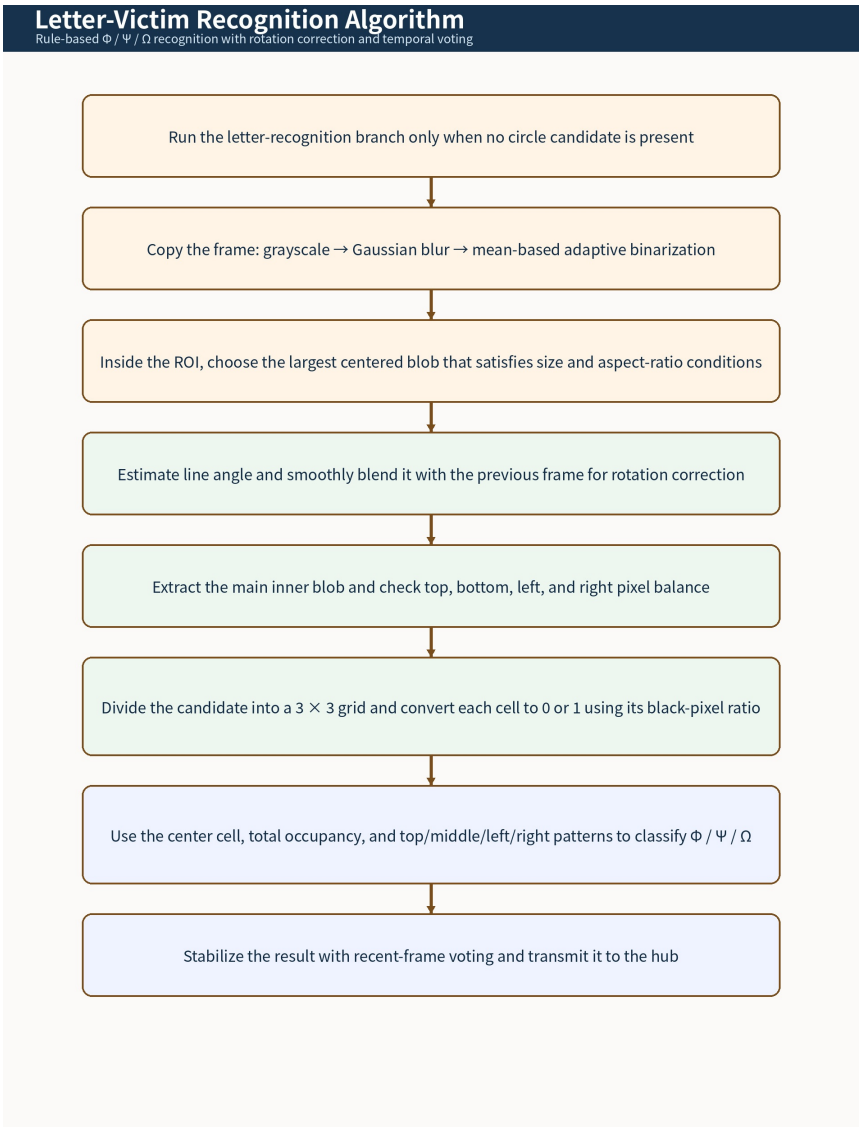

Figure S4. Rotation-corrected, 3 x 3 pattern-based recognition of Phi, Psi, and Omega.

Letter victim	Classification value	Rescue kits
Omega (OMEGA)	0	0
Psi (PSI)	1	1
Phi (PHI)	2	2

3. Asynchronous Sensor Processing and Rescue-Kit Deployment

The camera board must recognize victims, read three ToF sensors, update communication values, and control the kit dispenser. Performing these actions sequentially would introduce avoidable delays. To solve this integration problem, the OpenMV program runs separate uasyncio tasks for vision, ToF polling, sensor updates, and queued dispenser motion. The Hub sends only the required kit command and continues navigation logic while OpenMV manages the physical release sequence.

Task or mechanism	Current implementation	Benefit
Vision processing	Continuous OpenMV frame processing with lens correction and rule-based recognition.	Recognition remains independent of Hub driving logic.
ToF acquisition	Adaptive polling interval from 10 ms to 80 ms with recovery handling.	Temporary sensor delays do not permanently stop the system.
Communication update	Compact PUPRemote values channel.	The Hub receives recognition, ToF, and touch data through one interface.
Dispenser control	Bounded command queue with maximum length 4 and a separate drop task.	Kit deployment does not unnecessarily block camera or ranging updates.

4. Adaptive Exploration Instead of a Simple Right-Hand Rule

The route selector keeps the intuitive right-forward-left-back preference but augments it with map information. For each possible direction, it compares exit-use counts and neighboring-tile visit counts. Fully exhausted branches are marked as dead ends and propagated through the map. When exploration is complete, the robot switches to BFS-based return navigation. This combination keeps the program lightweight while reducing avoidable revisits.

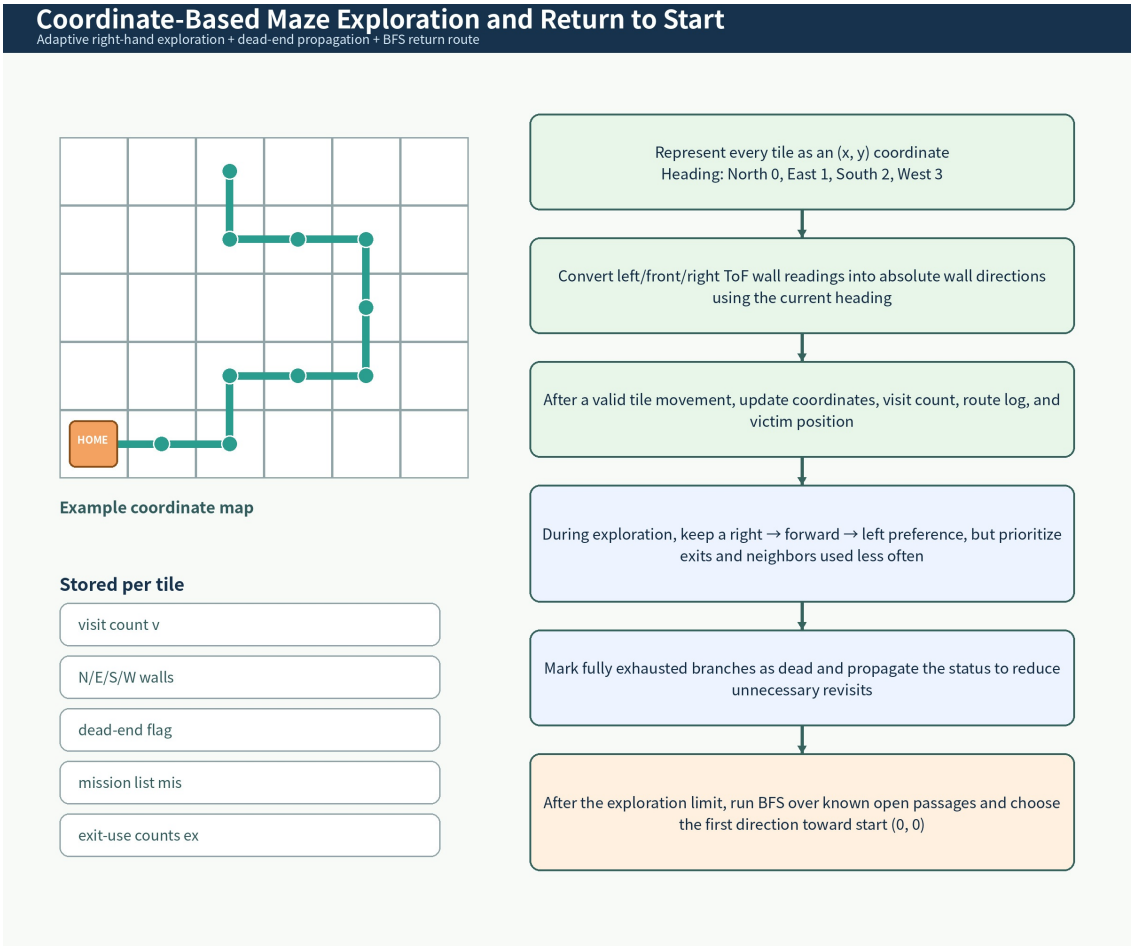

Figure S5. Coordinate-based exploration with dead-end propagation and BFS return-to-start navigation.

5. Validation Procedures and Current Technical Data

Validation was performed by isolating subsystems before integration and then combining them into the full OpenMV-Hub workflow. The table separates a measured speed result from implemented or calibrated parameters. Additional full-course success-rate data can be inserted after repeated field runs.

Validation item	Procedure	Current result or parameter	Type
OpenMV circle-detection speed	Run the circle-recognition loop with timing output enabled.	Approximately 76 frames per second (about 13 ms per frame).	Measured speed test
Ring-color classification	Sample five radial zones and classify pixels in 16 directions for each zone.	80 LAB samples per accepted target frame.	Implemented algorithm
Letter stabilization	Store recent recognition outputs and vote before sending the result.	Recent-result window: 5 frames.	Implemented parameter
Target position filtering	Reject candidates outside the central acceptance window.	Center (80, 60), tolerance +/-30 px horizontally and +/-45 px vertically.	Implemented parameter
ToF sensor scheduling	Poll sensors continuously and adapt interval when recovery is required.	Adaptive interval: 10-80 ms.	Implemented parameter
Dispenser calibration	Tune left and right motion durations and include a stop interval.	Right drop/return: 382/389 ms; left drop/return: 402/399 ms; stop: 120 ms.	Calibrated parameter
Command buffering	Send multiple deployment commands while monitoring queue state.	Maximum pending queue length: 4 commands.	Implemented parameter

Testing Sequence

Stage	Test method	Pass criterion
Vision-only test	Present cognitive targets and letter victims at different angles and positions while monitoring recognition output.	Correct class remains stable across consecutive frames and processing remains real-time.
Sensor-only test	Read left/front/right ToF values and touch states while moving objects toward each sensing direction.	Distances and touch states update without persistent communication failure.
Communication test	Read the values channel on the Hub and send dispenser commands back to OpenMV.	The Hub receives updated values and OpenMV reports queued commands.
Deployment test	Trigger left and right release sequences repeatedly.	A rescue kit is released to the requested side and the mechanism returns to neutral.
Navigation test	Run exploration, dead-end handling, and return logic on a mapped maze.	Coordinates, wall links, and victim records remain consistent; the robot calculates a return route to (0, 0).

Further data to add after full-course runs: cognitive-target accuracy by lighting condition, letter-victim accuracy by viewing angle, average detection distance, left/right kit-deployment success rate, return-to-start completion rate, and total autonomous run time. These values require recorded field trials and are not inferred in this document.